Laboratório 05

Primeiro Contacto com Kubernetes

Infraestruturas de Computação na Nuvem | Aula 05

Objetivo e enquadramento

A Aula 05 apresenta, no plano teórico, o modelo de funcionamento do Kubernetes: um conjunto de nós, objetos que descrevem o estado pretendido e controladores que o mantêm. Este laboratório torna esse modelo observável num cluster criado no computador do próprio aluno.

O trabalho não requer acesso ao servidor da unidade curricular nem a qualquer serviço de nuvem. Destina-se a ser realizado durante a aula, ao ritmo de cada aluno, e **não constitui trabalho a entregar**. Pressupõe apenas que o Docker está instalado e em funcionamento, tal como no laboratório da Aula 03.

O laboratório utiliza o `kind` (*Kubernetes in Docker*), que cria cada nó do cluster como um container Docker. Trata-se de uma solução adequada a fins pedagógicos, por exigir apenas o Docker e por reutilizar o que foi estudado na Aula 04.

Resultados de aprendizagem

No final deste laboratório, o aluno deve ser capaz de:

- Criar um cluster Kubernetes local e identificar os seus nós.
- Descrever uma aplicação num ficheiro YAML e aplicá-la com o `kubectl`.
- Relacionar o Deployment, o ReplicaSet e os Pods que dele resultam.
- Observar o ciclo de reconciliação, apagando um Pod e verificando a sua substituição.
- Expor a aplicação através de um Service e explicar a função das etiquetas.

- Atualizar a versão de uma aplicação e regressar à versão anterior.

Requisitos

- Computador com Windows, macOS ou Linux.
- Docker instalado e em execução (Docker Desktop em Windows e macOS).
- Cerca de 4 GB de memória atribuídos ao Docker e alguns gigabytes de espaço livre em disco.
- Ligação à Internet, necessária para obter as imagens.
- Duração estimada: entre 60 e 90 minutos.

Como utilizar este guião

Cada passo apresenta, por esta ordem, o objetivo, os comandos a executar e o resultado esperado. Recomenda-se ler o passo completo antes de executar o primeiro comando. As caixas têm sempre o mesmo significado:

Comando a executar

```
Comando a executar no terminal.
```

Resultado esperado (os valores podem variar)

```
Resultado que deve ser apresentado.
```

Analisar

Momento de pausa para interpretar o resultado obtido, isoladamente ou em discussão com os colegas.

Erro possível e resolução

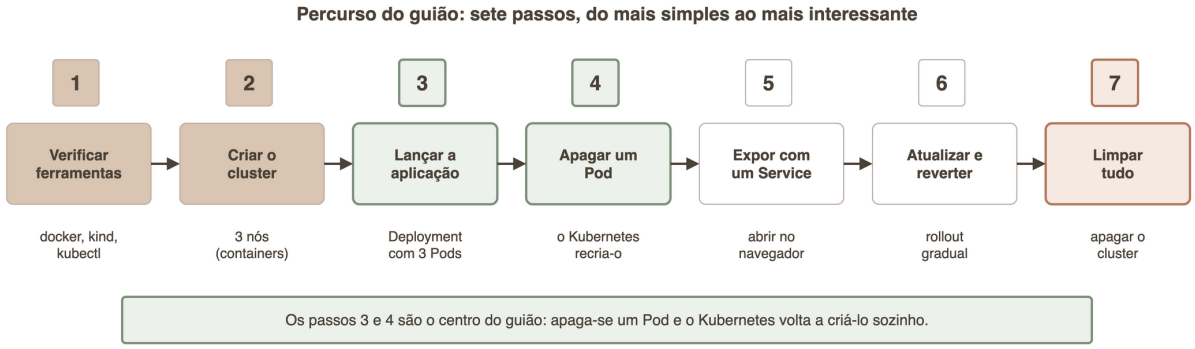
Erro que pode ocorrer e respetiva resolução.

Informação complementar que facilita a execução.

Os comandos podem ser copiados e colados. Sempre que um valor surja entre os sinais de menor e maior, como `<nome-do-pod>`, deve ser substituído, incluindo os sinais, pelo valor real apresentado no ecrã. Os resultados mostrados são exemplos: nomes de Pods, endereços IP, tempos e números de versão diferem de computador para computador.

Percurso

A figura seguinte resume a sequência do guião. Os passos 3 e 4 constituem a observação central: o Kubernetes restabelece, por si, o estado que foi declarado.



Como ler a figura

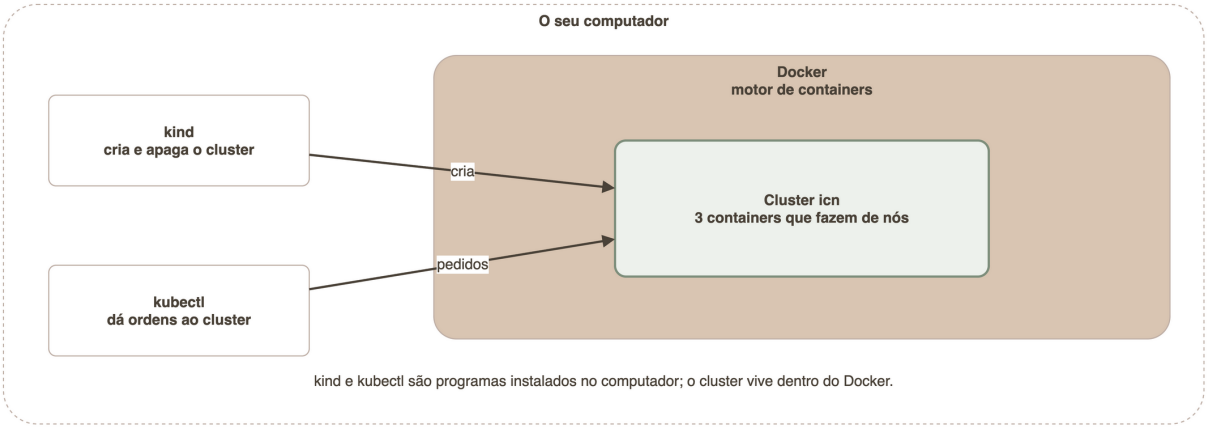
Os sete passos leem-se da esquerda para a direita. Os dois primeiros, a bege, preparam o ambiente: verificar as ferramentas e criar o cluster. Os passos 3 e 4, a verde, são a parte central: declara-se uma aplicação e observa-se a reação do Kubernetes quando um Pod é removido. Os passos 5 e 6, a branco, acrescentam o acesso à aplicação e a sua atualização. O passo 7, em tom avermelhado, encerra o trabalho e liberta os recursos. Abaixo de cada caixa, uma legenda resume o que é feito nesse passo.

Passo 1 — Verificação das ferramentas

Objetivo. Confirmar que o Docker, o `kind` e o `kubectl` estão instalados e a funcionar.

Enquadramento. O `kind` cria o cluster utilizando containers Docker como nós. O `kubectl` é a ferramenta de linha de comandos através da qual se comunica com o cluster.

A figura mostra a relação entre as três ferramentas: o `kind` e o `kubectl` são programas instalados no computador, e o cluster que o `kind` cria existe dentro do Docker.



Como ler a figura

À esquerda estão os dois programas que o aluno utiliza: o `kind`, que cria e apaga o cluster, e o `kubectl`, que envia pedidos a um cluster já criado. À direita, a caixa maior representa o Docker e, dentro dela, a caixa verde representa o cluster, formado por três containers. A seta do `kind` indica que a criação é feita através do Docker. A seta do `kubectl` indica que os pedidos seguem diretamente para o cluster. Em resumo, as ferramentas ficam fora e o cluster vive dentro do Docker.

Fundamento teórico

O Kubernetes é, antes de mais, uma API. O `kubectl` é um cliente que traduz cada comando num pedido HTTP ao API server do cluster e apresenta a resposta. Para saber a que cluster falar e com que credenciais, lê o ficheiro kubeconfig, normalmente em `~/.kube/config`. Esse ficheiro guarda contextos. Cada contexto associa um cluster, um utilizador e, opcionalmente, um espaço de nomes. Ao criar um cluster, o `kind` acrescenta o contexto `kind-icn` e ativa-o. Por isso o comando `kubectl config use-context` resolve o erro de ligação recusada.

Os nós do `kind` não são uma versão reduzida do Kubernetes. São containers de sistema, no sentido da Aula 04: executam o `systemd` e, no seu interior, os mesmos componentes de

um nó real, isto é, o kubelet, o `containerd` e, no nó de controlo, o API server, o etcd, o scheduler e o controller manager. Partilham o núcleo do anfitrião. Três nós custam, assim, muito menos memória do que três máquinas virtuais. Em Windows e macOS, esse núcleo é o da máquina virtual Linux do Docker Desktop.

A analisar neste passo: quem fala com quem. O `kind` e o `kubectl` são programas distintos, e só o segundo comunica com o cluster depois de este existir.

1.1 Abertura do terminal

Em Windows, abrir o **PowerShell**. Em macOS, abrir o **Terminal**. Em Linux, abrir o terminal da distribuição.

1.2 Criação da pasta de trabalho

Os dois ficheiros deste guião são guardados numa pasta própria.

Comando a executar

```
mkdir icn-aula05
cd icn-aula05
```

1.3 Verificação do Docker

Comando a executar

```
docker --version
docker ps
```

Resultado esperado (os valores podem variar)

```
Docker version 27.x.x, build abcdef1
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
```

A lista de containers pode estar vazia. O essencial é que os comandos não devolvam erro.

Erro possível e resolução

Cannot connect to the Docker daemon ou **error during connect**: o Docker não está em execução. Abrir o Docker Desktop e aguardar o estado *running*. Em Linux, executar `sudo systemctl start docker`.
permission denied while trying to connect (Linux): o utilizador não pertence ao grupo `docker`. Executar `sudo usermod -aG docker $USER`, terminar a sessão e iniciá-la de novo.
docker: command not found: o Docker não está instalado. Instalar o Docker Desktop (Windows e macOS) ou o Docker Engine (Linux), conforme https://docs.docker.com/get-docker/.

1.4 Instalação do kind e do kubectl

Deve ser seguida **apenas** a indicação correspondente ao sistema operativo em uso.

macOS (com Homebrew):

Comando a executar

```
brew install kind kubectl
```

Windows (PowerShell):

Comando a executar

```
winget install Kubernetes.kind
winget install Kubernetes.kubectl
```

No final, fechar o PowerShell e abrir outro, para que os comandos passem a ser reconhecidos.

Linux: seguir as instruções oficiais em https://kind.sigs.k8s.io/docs/user/quick-start/ e em https://kubernetes.io/docs/tasks/tools/, que descrevem a obtenção de um único ficheiro executável.

1.5 Confirmação da instalação

Comando a executar

```
kind version
kubectl version --client
```

Resultado esperado (os valores podem variar)

```
kind v0.xx.x go1.xx.x darwin/arm64
Client Version: v1.xx.x
Kustomize Version: v5.x.x
```

Erro possível e resolução

kind: command not found ou termo não reconhecido: fechar o terminal e abrir um novo. Se persistir, a pasta de instalação não consta da variável PATH; repetir a instalação e ler a mensagem final.
brew: command not found: instalar o Homebrew a partir de `https://brew.sh` ou usar a obtenção direta descrita nas páginas oficiais.

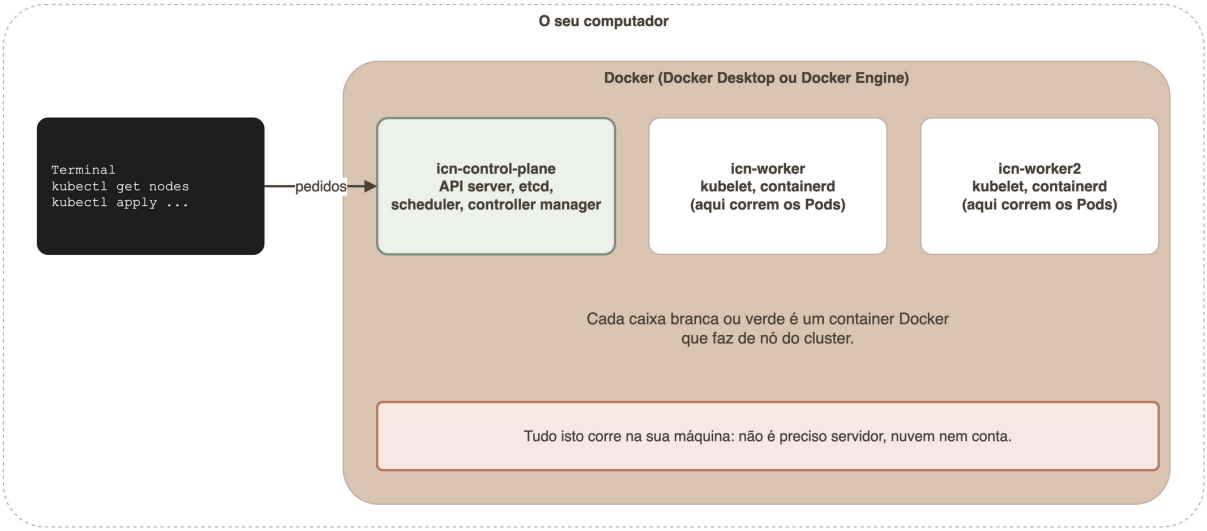
Verificação antes de avançar

Os comandos `docker ps`, `kind version` e `kubectl version --client` executam sem erro.

Passo 2 — Criação do cluster

Objetivo. Descrever, num ficheiro, um cluster com três nós e solicitar ao `kind` a sua criação.

Enquadramento. Com um nó de controlo e dois nós de trabalho, é possível observar, mais adiante, a distribuição dos Pods por máquinas diferentes, tal como no diagrama da Aula 05.



Como ler a figura

A figura detalha o que existe dentro do Docker no final deste passo. O nó de controlo, a verde, aloja os componentes que decidem: o API server, o etcd, o scheduler e o controller manager. Os dois nós de trabalho, a branco, executam os Pods, por intermédio do kubelet e do containerd. O terminal à esquerda representa o `kubectl`, cujos pedidos chegam sempre ao API server. Cada caixa corresponde a um dos três containers apresentados por `docker ps`.

Fundamento teórico

Um cluster divide-se em plano de controlo e nós de trabalho. No nó de controlo correm os componentes que decidem. Nos nós de trabalho correm as aplicações. Por omissão, o nó de controlo tem uma marca, um *taint*, que impede o scheduler de lá colocar Pods comuns. É por isso que, neste laboratório, os Pods aparecem apenas nos dois nós de trabalho.

O estado `Ready` não significa apenas que a máquina está ligada. O kubelet comunica periodicamente com o API server e o nó só é declarado pronto quando o *runtime* de containers funciona e a rede de Pods está configurada. Sem plugin de rede, o nó fica `NotReady`. O `kind` instala um plugin por omissão, e por isso o estado `Ready` surge sem passos adicionais.

A criação segue a sequência de um cluster real. O `kind` prepara os containers, inicia o nó de controlo com o `kubeadm` e junta depois os nós de trabalho. É a mesma ferramenta de arranque do diagrama de instalação apresentado na aula.

A analisar neste passo: a correspondência entre nós do Kubernetes e containers do Docker, e a diferença de papéis entre o nó de controlo e os restantes.

2.1 Ficheiro de configuração

Criar, dentro da pasta `icn-aula05`, um ficheiro com o nome `kind-icn.yaml` e o conteúdo seguinte. Pode ser utilizado o Bloco de Notas (Windows: `notepad kind-icn.yaml`), o `nano` (macOS e Linux: `nano kind-icn.yaml`) ou outro editor de texto simples.

Conteúdo do ficheiro `kind-icn.yaml`

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
  - role: control-plane
  - role: worker
  - role: worker
```

Erro possível e resolução

Os ficheiros YAML usam espaços, e não tabulações, para a indentação. Devem manter-se exatamente dois espaços antes de cada `- role`. Se o ficheiro for gravado como `kind-icn.yaml.txt`, o `kind` não o encontra: apagá-lo e gravar de novo com o nome correto.

2.2 Criação do cluster

A primeira execução demora entre dois e cinco minutos, por incluir a obtenção da imagem dos nós.

Comando a executar

```
kind create cluster --name icn --config kind-icn.yaml
```

Resultado esperado (os valores podem variar)

```
Creating cluster "icn" ...
 Ensuring node image (kindest/node:v1.xx.x)
 Preparing nodes
 Writing configuration
 Starting control-plane
 Installing CNI
 Installing StorageClass
 Joining worker nodes
Set kubectl context to "kind-icn"
You can now use your cluster with:

kubectl cluster-info --context kind-icn
```

Algumas linhas reais apresentam pequenos símbolos à esquerda, omitidos neste exemplo.

Erro possível e resolução

node(s) already exist for a cluster with the name "icn": o cluster já foi criado anteriormente. Executar `kind delete cluster --name icn` e repetir a criação.
timed out waiting for the condition, ou execução aparentemente parada: o Docker dispõe de pouca memória. Em Docker Desktop, abrir *Settings*, depois *Resources*, e atribuir pelo menos 4 GB. Aplicar, aguardar o reinício do Docker, apagar o cluster com o comando anterior e repetir.
failed to pull image, ou erros de rede: verificar a ligação à Internet e repetir. Em redes institucionais, pode ser necessário utilizar outra rede.

2.3 Listagem dos nós

Comando a executar

```
kubectl get nodes
```

Resultado esperado (os valores podem variar)

```
NAME                 STATUS   ROLES           AGE   VERSION
icn-control-plane    Ready    control-plane   75s   v1.xx.x
icn-worker           Ready    <none>          55s   v1.xx.x
icn-worker2          Ready    <none>          55s   v1.xx.x
```

Se algum nó apresentar o estado **NotReady**, aguardar cerca de meio minuto e repetir o comando.

Erro possível e resolução

The connection to the server localhost:8080 was refused ou **context not found**: o `kubectl` não está associado ao cluster. Executar `kubectl config use-context kind-icn` e repetir.

2.4 Verificação de que os nós são containers

Comando a executar

```
docker ps
```

Resultado esperado (os valores podem variar)

```
CONTAINER ID   IMAGE                  STATUS         NAMES
3f1a9c2b7d10   kindest/node:v1.xx.x   Up 2 minutes   icn-worker2
8b7e6a5d4c31   kindest/node:v1.xx.x   Up 2 minutes   icn-worker
c0d9e8f7a6b5   kindest/node:v1.xx.x   Up 2 minutes   icn-control-plane
```

Analisar

O número de containers coincide com o número de nós listados no passo anterior. Cada nó do Kubernetes é, neste ambiente, um container Docker.
Recorde-se da Aula 04 que um container é um processo isolado por namespaces e limitado por cgroups. O cluster agora criado corre, portanto, sobre esse mesmo mecanismo.

Verificação antes de avançar

O comando `kubectl get nodes` apresenta três nós com o estado **Ready**.

Passo 3 — Execução de uma aplicação

Objetivo. Solicitar ao Kubernetes que mantenha três réplicas de um servidor web em execução.

Enquadramento. Trata-se do modo declarativo estudado na aula: não se indica como criar os Pods, mas qual o estado pretendido, cabendo ao Kubernetes atingi-lo e conservá-lo.

Fundamento teórico

Todo o objeto do Kubernetes tem a mesma estrutura de base. O campo `apiVersion` indica a versão da API, `kind` o tipo, `metadata` identifica o objeto e `spec` descreve o estado pretendido. Depois de criado, o objeto ganha um campo `status`, escrito pelo sistema, que regista o estado observado. A diferença entre `spec` e `status` é aquilo que os controladores procuram anular.

Um Deployment não cria Pods diretamente. Cria um ReplicaSet, cujo nome inclui um resumo do modelo de Pod, e é este que cria os Pods, com um sufixo aleatório. Daí a forma dos nomes, como `web-7c5ddbdd8d-4k8xq`: nome do Deployment, resumo do modelo, identificador do Pod. O `selector` e as etiquetas do modelo têm de coincidir, porque é assim que o ReplicaSet reconhece os Pods como seus.

O scheduler escolhe o nó de cada Pod em duas fases. Filtra os nós que não servem e pontua os restantes. Registada a escolha, o kubelet do nó obtém a imagem e inicia o container. A indicação `READY 1/1` significa que o único container do Pod está pronto.

O comando `kubectl apply` é idempotente. Aplicar o mesmo ficheiro duas vezes produz o mesmo resultado que aplicá-lo uma.

A analisar neste passo: a cadeia Deployment, ReplicaSet, Pod, e o facto de ter sido declarado um número e não uma sequência de ações.

3.1 Ficheiro da aplicação

Criar, na mesma pasta, um ficheiro com o nome `web.yaml`.

Conteúdo do ficheiro `web.yaml`

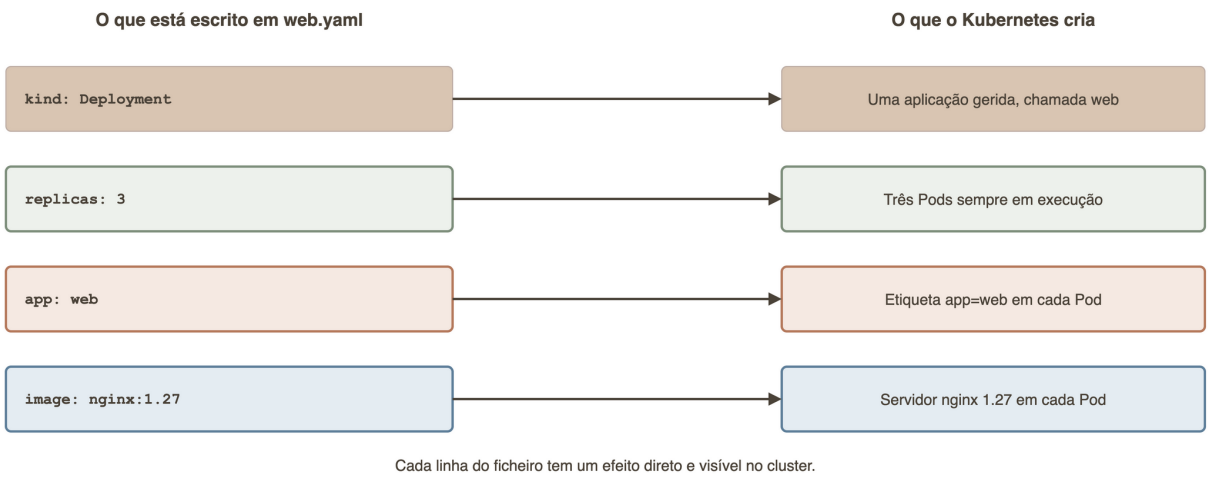
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
spec:
  replicas: 3
  selector:
```

```
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
        - name: nginx
          image: nginx:1.27
          ports:
            - containerPort: 80
```

O significado dos campos principais é o seguinte:

- `kind: Deployment`: objeto que descreve uma aplicação gerida, aqui designada `web`.
- `replicas: 3`: número de cópias que devem existir em permanência.
- `app: web`: etiqueta atribuída a cada cópia, que permite ao Kubernetes reconhecê-las como pertencentes à aplicação.
- `image: nginx:1.27`: imagem do servidor web nginx, na versão 1.27, executada em cada cópia.

A figura associa cada linha relevante do ficheiro ao efeito que produz no cluster.



Como ler a figura

Cada linha do ficheiro, à esquerda, produz o efeito indicado à direita, com a mesma cor. A linha `kind: Deployment` determina o tipo de objeto. A linha `replicas: 3` define quantos Pods devem existir em permanência. A linha `app: web` é a etiqueta que cada Pod recebe, e é por ela que o ReplicaSet e, mais tarde, o Service reconhecem os Pods. A linha `image: nginx:1.27` indica a imagem executada em cada Pod. Alterar uma destas linhas e aplicar de novo o ficheiro altera o cluster em conformidade.

3.2 Aplicação do ficheiro

Comando a executar

```
kubectl apply -f web.yaml
```

Resultado esperado (os valores podem variar)

```
deployment.apps/web created
```

Erro possível e resolução

error converting YAML to JSON ou **mapping values are not allowed**: existe um erro de indentação ou um carácter em excesso. Comparar o ficheiro com o do guião linha a linha e confirmar a utilização de espaços, e não de tabulações.
error: the path "web.yaml"does not exist: o terminal não se encontra na pasta do ficheiro. Executar `pwd` para verificar a localização e `cd icn-aula05` para entrar na pasta.

3.3 Listagem dos Pods

Comando a executar

```
kubectl get pods -o wide
```

Resultado esperado (os valores podem variar)

```
NAME                   READY   STATUS    RESTARTS   AGE   IP           NODE
web-7c5ddbdd8d-4k8xq   1/1     Running   0          40s   10.244.1.2   icn-worker
web-7c5ddbdd8d-m2n7p   1/1     Running   0          40s   10.244.2.2   icn-worker2
```

```
web-7c5ddbdd8d-x9z4t   1/1     Running   0          40s   10.244.1.3   icn-worker
```

Se o estado for **ContainerCreating**, a imagem ainda está a ser obtida. Aguardar alguns segundos e repetir.

Erro possível e resolução

ImagePullBackOff ou **ErrImagePull**: não foi possível obter a imagem. A causa surge na secção *Events* do resultado de `kubectl describe pod <nome-do-pod>`. Corresponde, em regra, a falta de ligação à Internet ou a erro de escrita em `nginx:1.27`.
Pending durante muito tempo: normalmente, falta de memória ao Docker. Aumentá-la, conforme indicado no passo 2.

3.4 Hierarquia de objetos

Comando a executar

```
kubectl get deployment
kubectl get replicaset
```

Resultado esperado (os valores podem variar)

```
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    3/3     3            3           1m

NAME             DESIRED   CURRENT   READY   AGE
web-7c5ddbdd8d   3         3         3       1m
```

Analisar

Foi criado apenas um Deployment. Existe, contudo, um ReplicaSet com nome derivado do Deployment, e três Pods cujos nomes têm origem no nome desse ReplicaSet. Está assim visível a cadeia estudada na aula: Deployment, ReplicaSet, Pods.
Na coluna **NODE** do comando anterior, os Pods surgem repartidos pelos dois nós de trabalho. Essa decisão foi tomada pelo scheduler.

Verificação antes de avançar

Existem três Pods com o estado **Running** e a indicação **1/1**.

Passo 4 — Remoção de um Pod

Objetivo. Remover deliberadamente um dos Pods e observar a reação do Kubernetes.

Enquadramento. Este é o ponto central da aula. O Kubernetes compara continuamente o estado pretendido, três Pods, com o estado observado, e corrige qualquer diferença sem intervenção do utilizador.



Como ler a figura

A figura apresenta três instantes, da esquerda para a direita. No primeiro, o Deployment pretende três Pods e existem três. No segundo, imediatamente após o `kubectl delete pod`, o Pod B desapareceu e restam dois, o que contraria o estado pretendido. No terceiro, o ReplicaSet criou um Pod novo, designado B', e o número voltou a ser três. O Pod novo não é o antigo recuperado: tem outro nome e outro endereço IP.

Fundamento teórico

Um controlador é um ciclo que observa, compara e age. O do ReplicaSet observa os Pods que correspondem ao seu *selector*, conta-os e compara a contagem com `replicas`. Se faltam, cria. Se sobram, elimina. Repete-o continuamente e reage ao estado, não ao evento: mesmo que perdesse a notificação da remoção, a contagem seguinte revelaria a diferença.

Remover um Pod é pedir ao API server que o marque para eliminação. O kubelet envia ao processo o sinal de terminação, aguarda por omissão até trinta segundos e, se o processo não terminar, força-o. É a fase `Terminating`. Entretanto, o ReplicaSet já criou o substituto, porque um Pod em eliminação deixou de contar.

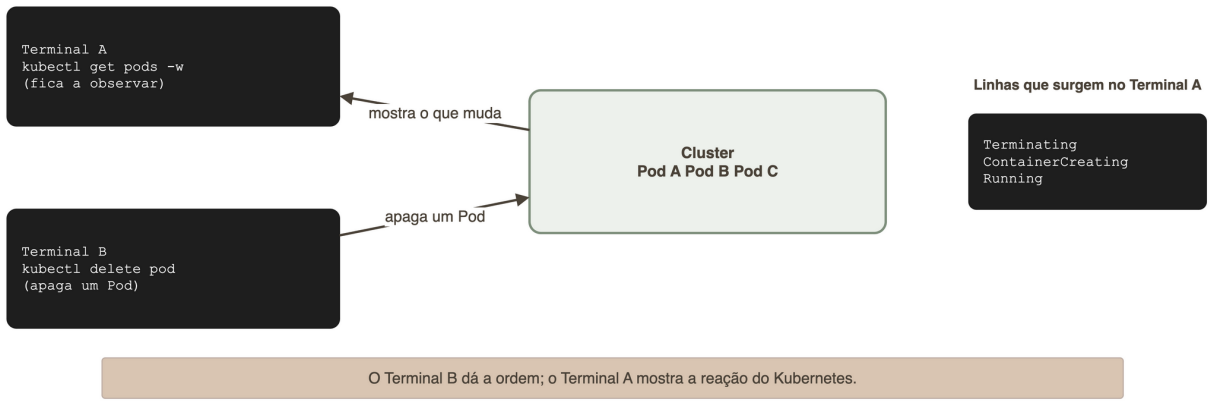
O Pod novo é outro objeto, com outro nome e outro endereço IP. O Kubernetes não ressuscita Pods, substitui-os. Um Pod nunca deve, por isso, ser tratado como uma máquina com identidade própria.

Cada Pod guarda uma referência ao ReplicaSet que o criou. Se o ReplicaSet ou o Deployment fossem removidos, os Pods seriam eliminados em cascata. Remover só o Pod não altera o estado pretendido e, por isso, não tem efeito duradouro. A alteração do número de réplicas, como no ponto 4.4, atua sobre o estado pretendido e é tratada pelo mesmo ciclo.

A analisar neste passo: a sequência de estados no Terminal A, a diferença de nome e de IP antes e depois, e a razão pela qual o número final de Pods é o declarado.

Para acompanhar o processo, são utilizados **dois terminais** em simultâneo.

A figura representa a disposição: o Terminal A apenas observa, e o Terminal B emite a ordem de remoção.



Como ler a figura

A figura mostra a divisão de tarefas entre os dois terminais. O Terminal A executa `kubectl get pods -w` e limita-se a observar, sem alterar nada. O Terminal B emite a ordem de remoção. A seta de B para o cluster representa essa ordem, e a seta do cluster para A representa as linhas que o observador vai recebendo. A caixa da direita lista os estados que devem surgir no Terminal A, pela ordem habitual: `Terminating`, `ContainerCreating` e `Running`.

4.1 Terminal A: observação

Abrir um novo terminal, entrar na pasta `icn-aula05` e executar:

Comando a executar

```
kubectl get pods -w
```

O comando permanece ativo e apresenta uma nova linha sempre que algo se altera. Deve ser mantido aberto.

Resultado esperado (os valores podem variar)

```
NAME                   READY   STATUS    RESTARTS   AGE
web-7c5ddbdd8d-4k8xq   1/1     Running   0          6m
web-7c5ddbdd8d-m2n7p   1/1     Running   0          6m
web-7c5ddbdd8d-x9z4t   1/1     Running   0          6m
```

4.2 Terminal B: remoção

No outro terminal, escolher o nome de um Pod da lista e removê-lo, substituindo `<nome-do-pod>` pelo nome real, por exemplo `web-7c5ddbdd8d-4k8xq`.

Comando a executar

```
kubectl delete pod <nome-do-pod>
```

Resultado esperado (os valores podem variar)

```
pod "web-7c5ddbdd8d-4k8xq" deleted
```

Para copiar o nome do Pod, basta selecioná-lo com duplo clique no terminal. Muitos terminais completam também o nome com a tecla Tab, após a escrita das primeiras letras.

4.3 Regresso ao Terminal A

Em poucos segundos surgem novas linhas:

Resultado esperado (os valores podem variar)

```
NAME                   READY   STATUS              RESTARTS   AGE
web-7c5ddbdd8d-4k8xq   1/1     Terminating         0          7m
web-7c5ddbdd8d-q7r2w   0/1     ContainerCreating   0          1s
web-7c5ddbdd8d-q7r2w   1/1     Running             0          3s
web-7c5ddbdd8d-4k8xq   0/1     Terminating         0          7m
```

Concluída a observação, premir **Ctrl+C** no Terminal A para libertar o terminal.

Analisar

Após a remoção de um Pod, voltam a existir três. O novo Pod não foi criado por solicitação do utilizador: foi o ReplicaSet que detetou a existência de dois Pods em vez de três e criou o que faltava.
O comando seguinte permite comparar o nome e o endereço IP do Pod novo com os do Pod removido.

Comando a executar

```
kubectl get pods -o wide
```

Ambos diferem. O Kubernetes não repara um Pod, substitui-o por outro. Por esta razão, um Pod não deve ser referido pelo seu endereço, e é essa a questão que o Service do passo seguinte resolve.

4.4 Variação do número de réplicas

O estado pretendido pode ser alterado com um único comando. Aumentar para cinco réplicas e observar o resultado:

Comando a executar

```
kubectl scale deployment web --replicas=5
kubectl get pods
```

Repor o valor original:

Comando a executar

```
kubectl scale deployment web --replicas=3
kubectl get pods
```

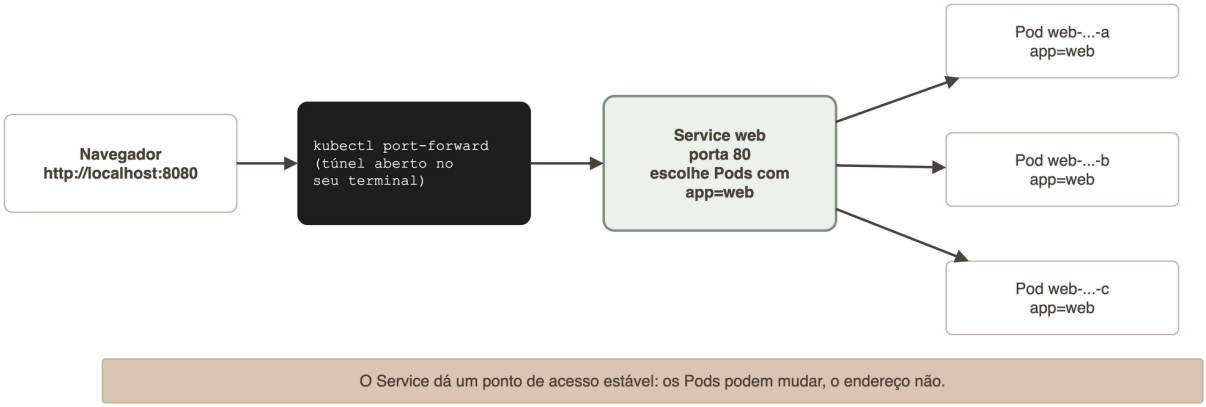
Verificação antes de avançar

É possível explicar por que razão, após a remoção de um Pod, voltaram a existir três.

Passo 5 — Exposição através de um Service

Objetivo. Atribuir um ponto de acesso estável aos três Pods e aceder-lhe a partir do navegador.

Enquadramento. Os Pods são efémeros e mudam de endereço quando são substituídos. O Service constitui um ponto de entrada que se mantém.



Como ler a figura

Lê-se da esquerda para a direita. O navegador acede ao endereço local `localhost:8080`. O comando `kubectl port-forward` mantém um túnel entre essa porta e um dos Pods do Service. O Service, que escuta na porta 80, seleciona os Pods com a etiqueta `app=web`, e o `kubectl` escolhe um deles quando abre o túnel. O túnel existe apenas para efeitos do laboratório, porque o Service é do tipo ClusterIP e só é acessível no interior do cluster. Os nomes dos Pods na figura são ilustrativos.

Fundamento teórico

Um Pod tem um endereço IP que muda quando o Pod é substituído. O Service resolve o problema com um endereço virtual estável, o ClusterIP, retirado de uma gama reservada ao cluster. Esse endereço não pertence a nenhuma interface. É implementado em cada nó pelo kube-proxy, que instala regras de rede a reescrever o destino para o endereço de um dos Pods.

A lista de destinos não é escrita à mão. Um controlador procura os Pods que satisfazem o *selector* do Service e que estão prontos, e regista os seus endereços em objetos *EndpointSlice*. Um Pod que falhe a sonda de *readiness* sai da lista. Dentro do cluster, o Service pode ainda ser alcançado pelo nome, `web`, graças ao CoreDNS.

O tipo ClusterIP não é acessível do exterior. Existem o NodePort e o LoadBalancer para esse fim, mas aqui usa-se o `port-forward`, que é uma ferramenta de desenvolvimento. O `kubectl` liga-se ao API server, que reencaminha o tráfego até ao kubelet e daí ao Pod. Ao

indicar um Service, o `kubectl` escolhe um dos Pods no momento em que abre o túnel, e todo o tráfego desse túnel segue para esse Pod. Atualizar a página não distribui, portanto, os pedidos pelos três.

A analisar neste passo: a correspondência entre os *Endpoints* e os IP dos Pods, e o papel das etiquetas na ligação entre Service e Pods.

5.1 Criação do Service

Comando a executar

```
kubectl expose deployment web --port=80
```

Resultado esperado (os valores podem variar)

```
service/web exposed
```

Comando a executar

```
kubectl get service web
```

Resultado esperado (os valores podem variar)

```
NAME   TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
web    ClusterIP   10.96.123.45   <none>        80/TCP    10s
```

O tipo é **ClusterIP**, pelo que o Service apenas é acessível no interior do cluster. Para lhe aceder a partir do navegador, é aberto um túnel temporário.

5.2 Abertura do túnel

Comando a executar

```
kubectl port-forward service/web 8080:80
```

Resultado esperado (os valores podem variar)

```
Forwarding from 127.0.0.1:8080 -> 80
Forwarding from [::1]:8080 -> 80
```

O terminal fica ocupado enquanto o túnel estiver aberto, o que é o comportamento esperado. **Não deve ser fechado.**

Erro possível e resolução

address already in use ou **unable to listen on port 8080**: outra aplicação usa a porta 8080. Escolher outra, por exemplo `kubectl port-forward service/web 8081:80`, e aceder então a `http://localhost:8081`.
unable to forward port because pod is not running: os Pods ainda estão a arrancar. Aguardar e repetir.
services "web"not found: o Service não foi criado. Voltar ao ponto 5.1.

5.3 Acesso pelo navegador

Com o túnel aberto, introduzir no navegador o endereço:

Comando a executar

```
http://localhost:8080
```

Deve ser apresentada a página **Welcome to nginx!**. Concluído o acesso, premir **Ctrl+C** no terminal do túnel.

Erro possível e resolução

A página não carrega: confirmar que o terminal do túnel continua aberto e a apresentar `Forwarding from`. Caso tenha sido fechado, repetir o ponto 5.2.

5.4 Destinos do tráfego

macOS e Linux:

Comando a executar

```
kubectl describe service web | grep Endpoints
```

Windows (PowerShell):

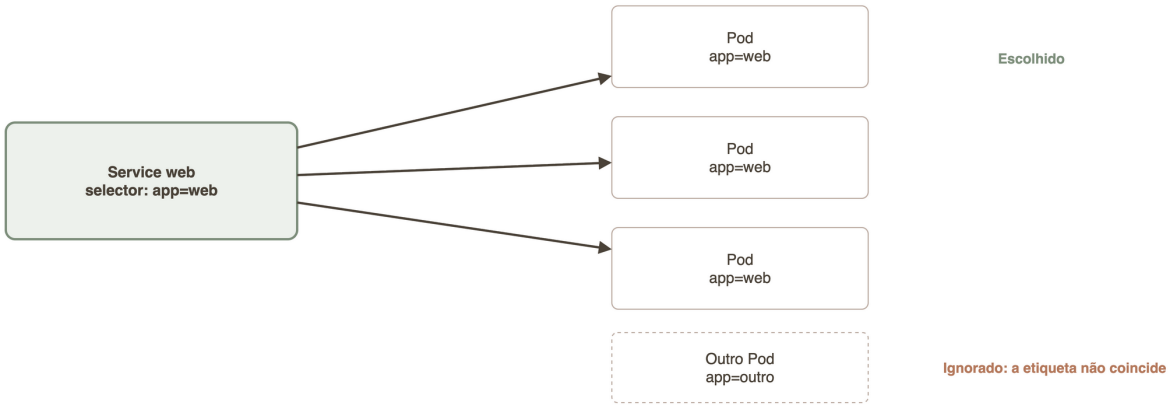
Comando a executar

```
kubectl describe service web | Select-String Endpoints
```

Resultado esperado (os valores podem variar)

```
Endpoints:  10.244.1.3:80,10.244.2.3:80,10.244.1.4:80
```

A figura seguinte ilustra a seleção por etiquetas: o Service escolhe os Pods cuja etiqueta coincide com o seu *selector* e ignora os restantes.



Como ler a figura

O Service não guarda uma lista de endereços. Guarda um critério, o *selector* `app=web`, e seleciona em cada momento os Pods cuja etiqueta o satisfaz. Os três Pods com `app=web` são escolhidos. O Pod com `app=outro` é ignorado, mesmo estando no mesmo cluster. Se surgir um Pod novo com `app=web`, passa a ser escolhido, e se um desaparecer, deixa de o ser. É por esta razão que a lista de *Endpoints* acompanha os Pods.

Analisar

A lista contém três endereços, correspondentes aos da coluna **IP** do comando `kubectl get pods -o wide`, isto é, aos três Pods.
O Service não mantém uma lista escrita manualmente: seleciona os Pods pela etiqueta `app=web`. Quando um Pod desaparece, o seu endereço sai da lista e o do substituto entra.

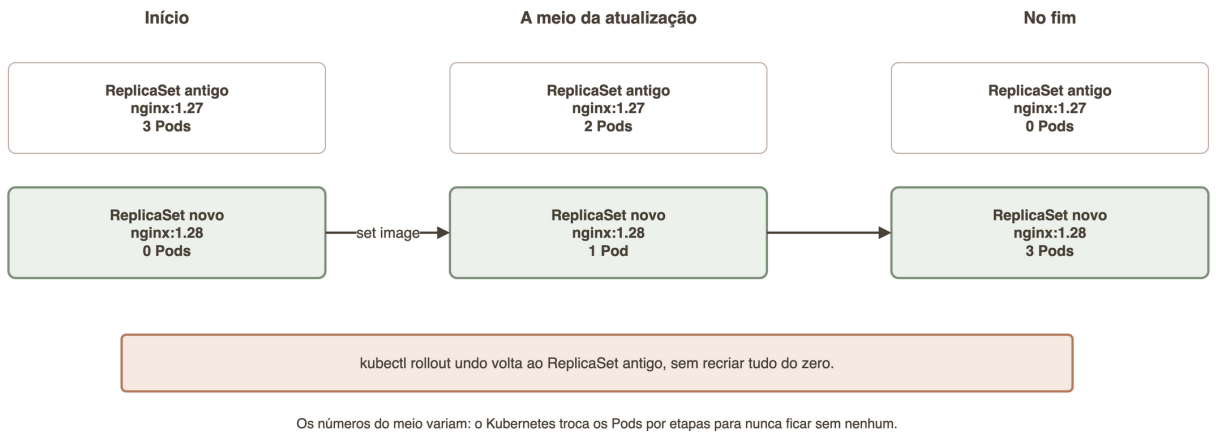
Verificação antes de avançar

A página do nginx foi apresentada no navegador e é compreendida a origem da lista de `Endpoints`.

Passo 6 — Atualização e reversão

Objetivo. Substituir a versão do nginx de 1.27 para 1.28 sem interromper o serviço, e regressar à versão anterior.

Enquadramento. Cada versão de uma aplicação tem o seu ReplicaSet. O Kubernetes transita de um para o outro por etapas, mantendo sempre capacidade disponível.



Como ler a figura

As três colunas representam três instantes da atualização. No início, o ReplicaSet antigo, com `nginx:1.27`, tem os três Pods e o novo, com `nginx:1.28`, não tem nenhum. A meio, o Kubernetes já criou um Pod da versão nova e retirou um da antiga, de modo que o serviço nunca fica sem Pods. No fim, todos os Pods são da versão nova e o ReplicaSet antigo ficou com zero. Este é conservado, e é por isso que o `kubectl rollout undo` é rápido. Os valores da coluna central são um exemplo e podem diferir.

Fundamento teórico

A atualização gradual é a estratégia por omissão do Deployment. O Kubernetes cria um ReplicaSet novo e transfere os Pods por etapas, reguladas por dois parâmetros: o número de Pods que podem existir acima do desejado, `maxSurge`, e o número que pode estar indisponível, `maxUnavailable`. Ambos valem por omissão vinte e cinco por cento, com arredondamento para cima no primeiro e para baixo no segundo. Com três réplicas, isto significa um Pod extra e nenhum indisponível. Cria-se um Pod novo, espera-se que fique pronto e só então se retira um antigo.

Sem sondas de *readiness*, um Pod é considerado pronto assim que o container arranca. Em aplicações reais, a sonda impede que o tráfego chegue a um Pod ainda não preparado.

Os ReplicaSets antigos ficam, sem Pods, até um limite de dez revisões por omissão. A

reversão não reconstrói nada: repõe no Deployment o modelo de Pod de uma revisão anterior.

Uma nota sobre o modo imperativo. O comando `kubectl set image` altera o objeto no cluster, mas não o ficheiro `web.yaml`, que continua a indicar a versão 1.27. Em contexto real, a alteração faz-se no ficheiro, que é a fonte de verdade, e aplica-se de novo.

A analisar neste passo: o número de Pods de cada ReplicaSet durante a atualização, e a diferença entre alterar o cluster por comando e alterar o ficheiro.

6.1 Atualização

Comando a executar

```
kubectl set image deployment/web nginx=nginx:1.28
kubectl rollout status deployment/web
```

Resultado esperado (os valores podem variar)

```
deployment.apps/web image updated
Waiting for deployment "web" rollout to finish: 1 out of 3 new replicas have been updated
    ...
Waiting for deployment "web" rollout to finish: 2 out of 3 new replicas have been updated
    ...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending termination...
deployment "web" successfully rolled out
```

Erro possível e resolução

unable to find container named "nginx": o nome anterior ao sinal de igual deve ser o do container indicado em `web.yaml`, isto é, `nginx`.
ImagePullBackOff após a atualização: a versão solicitada não existe ou não foi possível obtê-la. Regressar à versão anterior com o comando do ponto 6.3.

6.2 Os dois ReplicaSets

Comando a executar

```
kubectl get replicaset
```

Resultado esperado (os valores podem variar)

```
NAME             DESIRED   CURRENT   READY   AGE
web-7c5ddbdd8d   0         0         0       20m
web-6b8f9d4c5    3         3         3       1m
```

Analisar

Existem agora dois ReplicaSets: o antigo ficou sem Pods e o novo mantém três.
O ReplicaSet antigo não foi removido. Conserva a descrição da versão anterior, razão pela qual o regresso a essa versão é rápido.

6.3 Reversão

Comando a executar

```
kubectl rollout undo deployment/web
kubectl rollout status deployment/web
```

Resultado esperado (os valores podem variar)

```
deployment.apps/web rolled back
deployment "web" successfully rolled out
```

Confirmar a imagem em uso:

macOS e Linux:

Comando a executar

```
kubectl describe deployment web | grep Image
```

Windows (PowerShell):

Comando a executar

```
kubectl describe deployment web | Select-String Image
```

Resultado esperado (os valores podem variar)

```
    Image:        nginx:1.27
```

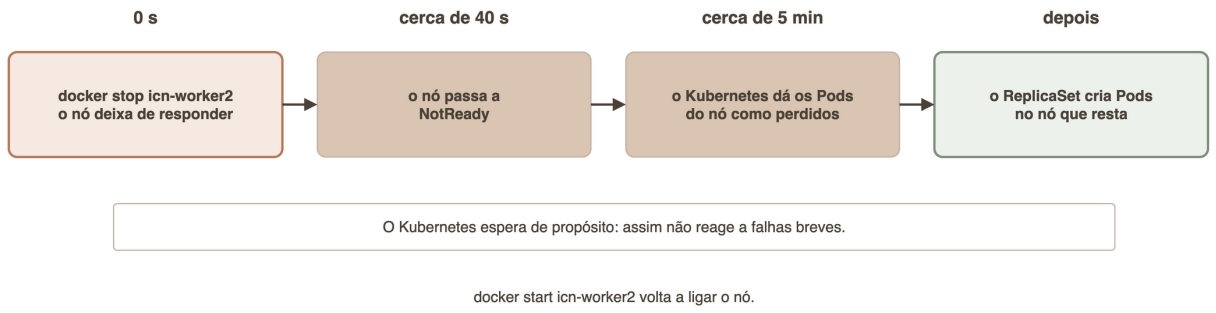
Verificação antes de avançar

A aplicação passou de `nginx:1.27` para `nginx:1.28` e regressou a `nginx:1.27`, sem remoção manual de qualquer objeto.

Desafio opcional: perda de um nó

Este desafio é facultativo, deve ser realizado apenas se o tempo o permitir e enquanto o cluster existir. Demora cerca de dez minutos.

A figura apresenta a sequência temporal esperada, desde a paragem do nó até à criação de Pods substitutos.



Como ler a figura

A sequência lê-se da esquerda para a direita. Ao parar o container do nó, este deixa de comunicar com o plano de controlo. Cerca de quarenta segundos depois, o nó é marcado como `NotReady`. Os Pods que nele estavam não são substituídos de imediato, porque o Kubernetes tolera, por omissão, cerca de cinco minutos de ausência antes de os dar como perdidos. Só então o ReplicaSet cria substitutos no nó que continua disponível. Os tempos são valores por omissão e podem variar.

Fundamento teórico

O kubelet de cada nó renova periodicamente, por omissão a cada dez segundos, um registo de presença no API server. Quando esses registos deixam de chegar durante cerca de quarenta segundos, o controlador de nós marca o nó como inacessível e o estado passa a `NotReady`. O controlador acrescenta então ao nó uma marca do tipo `NoExecute`, que repele Pods.

Os Pods recebem automaticamente uma tolerância a essa marca durante trezentos segundos. Só depois são expulsos, e só então o ReplicaSet cria substitutos noutro nó. Entretanto, o kubelet do nó parado não pode terminar os Pods, e o estado que o cluster conhece deles é o último que foi comunicado.

A espera é uma decisão de desenho. Reagir de imediato encurtaria a interrupção, mas faria o cluster tratar uma falha de rede de poucos segundos como uma avaria e deslocar cargas sem necessidade. Há também uma razão de correção. Sem saber se o nó está de facto parado, o Kubernetes arriscaria ter duas instâncias do mesmo Pod ativas ao mesmo tempo, o que é perigoso em aplicações com estado. Quem precisa de reagir mais depressa pode ajustar a tolerância por Pod.

A analisar neste desafio: os tempos observados e a relação entre disponibilidade e prudência.

1. Verificar em que nós se encontram os Pods.

Comando a executar

```
kubectl get pods -o wide
```

2. Parar um dos nós de trabalho, simulando a avaria da máquina.

Comando a executar

```
docker stop icn-worker2
```

3. Observar o estado dos nós.

Comando a executar

```
kubectl get nodes -w
```

Passado cerca de meio minuto, o estado do nó passa a **NotReady**.

Resultado esperado (os valores podem variar)

```
NAME                 STATUS     ROLES           AGE   VERSION
icn-control-plane    Ready      control-plane   30m   v1.xx.x
icn-worker           Ready      <none>          30m   v1.xx.x
icn-worker2          NotReady   <none>          30m   v1.xx.x
```

Analisar

Os Pods do nó parado não são recriados de imediato. Por omissão, o Kubernetes aguarda cerca de cinco minutos antes de os considerar perdidos, de modo a não reagir a falhas breves. Decorrido esse tempo, o ReplicaSet cria substitutos no nó que se mantém disponível. Premir Ctrl+C para sair do modo de observação e repetir `kubectl get pods -o wide` periodicamente.

4. Concluída a observação, voltar a iniciar o nó.

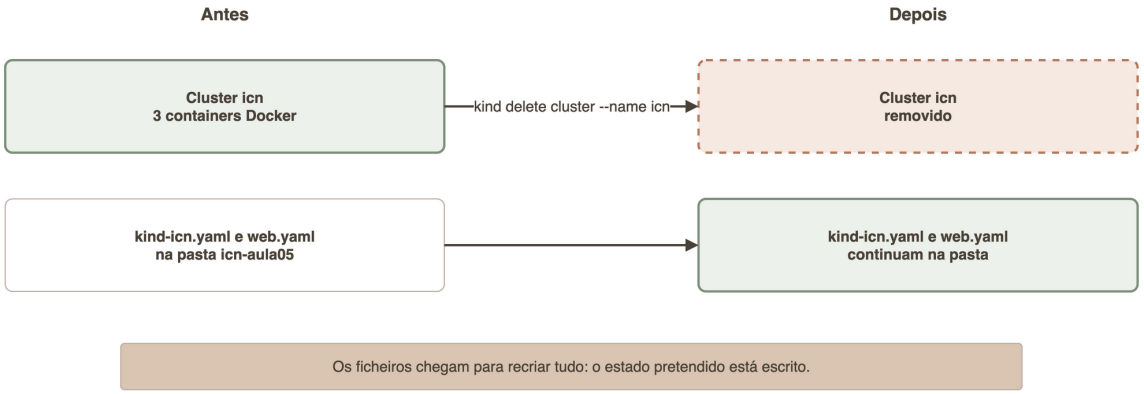
Comando a executar

```
docker start icn-worker2
```

Passo 7 — Remoção do cluster

Objetivo. Eliminar o cluster, libertando a memória e o espaço em disco utilizados.

A figura mostra o que é removido e o que permanece após a eliminação do cluster.



Como ler a figura

A figura compara o ambiente antes e depois do comando `kind delete cluster`. O cluster, composto por três containers Docker, é removido por completo, incluindo os Pods, os Deployments e os Services nele criados. Os dois ficheiros YAML, que estão numa pasta do computador e não dentro do cluster, permanecem. Com eles é possível recriar o mesmo ambiente com dois comandos, o que ilustra a vantagem de descrever o estado pretendido em ficheiros.

Fundamento teórico

O estado do cluster vive no etcd, que neste laboratório está dentro do container do nó de controlo. Eliminar o cluster elimina esses containers e, com eles, todos os objetos criados: Deployments, ReplicaSets, Pods e Services. Não há nada a desfazer objeto a objeto.

Sobrevive o que está fora do cluster. Os ficheiros `kind-icn.yaml` e `web.yaml` descrevem o cluster e a aplicação e bastam para os recriar. Esta é a ideia da infraestrutura descrita como código, que a Aula 07 aprofunda. O estado pretendido fica escrito, versionável e reproduzível, e o ambiente concreto passa a ser descartável.

A imagem dos nós fica na cache do Docker, o que acelera a criação seguinte. Pode ser removida com `docker image rm` se for preciso libertar espaço.

A analisar neste passo: o que deixou de existir e o que continua a permitir recriá-lo.

7.1 Eliminação

Comando a executar

```
kind delete cluster --name icn
```

Resultado esperado (os valores podem variar)

```
Deleting cluster "icn" ...
Deleted nodes: ["icn-worker2" "icn-worker" "icn-control-plane"]
```

7.2 Verificação

Comando a executar

```
docker ps
```

Resultado esperado (os valores podem variar)

```
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
```

Não deve surgir nenhum container `kindest/node`. Os ficheiros `kind-icn.yaml` e `web.yaml` permanecem na pasta e permitem reconstruir o ambiente em poucos minutos.

Para repetir o laboratório, bastam os comandos `kind create cluster --name icn --config kind-icn.yaml` e `kubectl apply -f web.yaml`. É esta a vantagem do modo declarativo: o estado pretendido fica escrito e é reproduzível.

Resolução de problemas: consulta rápida

Sintoma	Resolução
`Cannot connect to the Docker daemon`	Abrir o Docker Desktop e aguardar o estado *running*. Em Linux, `sudo systemctl start docker`.
`permission denied` (Docker, Linux)	`sudo usermod -aG docker $USER`, terminar a sessão e iniciá-la de novo.
`command not found` ou termo não reconhecido	Fechar e abrir o terminal. Se persistir, repetir a instalação da ferramenta.
`node(s) already exist ... "icn"`	`kind delete cluster --name icn` e repetir a criação.
Criação do cluster parada ou `timed out`	Atribuir pelo menos 4 GB de memória ao Docker, apagar o cluster e repetir.
`connection to the server ... refused`	`kubectl config use-context kind-icn`
Nó em `NotReady`	Aguardar meio minuto e repetir `kubectl get nodes`.
`error converting YAML to JSON`	Rever a indentação: apenas espaços, sem tabulações, igual ao guião.
`the path "web.yaml"does not exist`	Confirmar a pasta com `pwd` e entrar com `cd icn-aula05`.
`ContainerCreating` durante muito tempo	A imagem ainda está a ser obtida. Aguardar e repetir.
`ImagePullBackOff` ou `ErrImagePull`	`kubectl describe pod <nome>` e ler *Events*. Verificar a Internet e o nome da imagem.
Pod em `Pending`	Recursos insuficientes. Aumentar a memória do Docker.
`address already in use`	Usar outra porta local, por exemplo `8081:80`.
A página não abre no navegador	O terminal do `port-forward` tem de permanecer aberto.
Cluster ativo desconhecido	`kubectl config get-contexts`

Comandos utilizados

Comando	Finalidade
`kind create cluster --name icn --config kind-icn.yaml`	Criar o cluster de três nós
`kubectl get nodes`	Listar os nós
`kubectl apply -f web.yaml`	Aplicar o estado pretendido descrito no ficheiro
`kubectl get pods -o wide`	Listar Pods, com endereço IP e nó
`kubectl get pods -w`	Observar alterações em tempo real
`kubectl delete pod <nome>`	Remover um Pod
`kubectl scale deployment web --replicas=5`	Alterar o número de réplicas
`kubectl expose deployment web --port=80`	Criar um Service
`kubectl port-forward service/web 8080:80`	Abrir um túnel para o navegador
`kubectl describe <tipo> <nome>`	Apresentar o detalhe e os eventos de um objeto
`kubectl set image deployment/web nginx=nginx:1.28`	Atualizar a imagem
`kubectl rollout status deployment/web`	Acompanhar uma atualização
`kubectl rollout undo deployment/web`	Regressar à versão anterior
`kind delete cluster --name icn`	Eliminar o cluster

Síntese

O laboratório permitiu observar, na prática, as ideias centrais da Aula 05:

- Um **cluster** é um conjunto de nós; neste ambiente, cada nó era um container Docker.
- O **Deployment** declara o estado pretendido e o **ReplicaSet** mantém o número de Pods.
- A remoção de um Pod não o elimina de forma duradoura, porque o Kubernetes **reconcilia** o estado pretendido com o observado.
- Os Pods são efémeros, e o **Service** oferece-lhes um ponto de acesso estável, selecionando-os

por **etiquetas**.

- A **atualização gradual** transita entre ReplicaSets por etapas, e o **rollout undo** repõe a versão anterior.